

In-Memory vs Persistent Conversation History

1. Main Goal

We will compare two chatbot approaches:

Approach 1

InMemoryChatMessageHistory

Approach 2

FileChatMessageHistory

The main difference is:

In-Memory



History stays only while program is running

versus:

Persistent File-Based History



History can be loaded again after restart

2. Approach 1 — In-Memory Conversation History

This approach uses:

`InMemoryChatMessageHistory`

It stores chat messages inside application memory.

3. Complete In-Memory Chatbot Program

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import (
    ChatPromptTemplate,
    MessagesPlaceholder
)
```

```
from langchain_core.chat_history import (
    InMemoryChatMessageHistory
)
```

```
from langchain_core.runnables.history import (
```

```
RunnableWithMessageHistory
)

# =====

# CREATE MODEL

# =====

llm = ChatOpenAI(
    model="gpt-5.6-luna",
    api_key=os.getenv("OPENAI_API_KEY")
)

# =====

# CREATE PROMPT

# =====

prompt = ChatPromptTemplate.from_messages(
    [
```

```
(  
    "system",  
    ""  
  
    You are a helpful AI assistant.  
    Answer in simple English.  
    ""  
  
),  
  
MessagesPlaceholder(  
    variable_name="history"  
),  
  
(  
    "human",  
    "{question}"  
)  
]  
)
```

```
# =====
```

```
# CREATE LCEL CHAIN
```

```
# =====
```

```
chain = (  
    prompt  
    | llm  
)
```

```
# =====
```

```
# SESSION HISTORY STORE
```

```
# =====
```

```
store = {}
```

```
def get_session_history(session_id):
```

```
    if session_id not in store:
```

```
store[session_id] = (  
    InMemoryChatMessageHistory()  
)
```

```
return store[session_id]
```

```
# =====  
# HISTORY-AWARE RUNNABLE  
# =====
```

```
chatbot = RunnableWithMessageHistory(  
  
    chain,  
  
    get_session_history,  
  
    input_messages_key="question",
```

```
    history_messages_key="history"  
)
```

```
# =====
```

```
# SELECT SESSION
```

```
# =====
```

```
session_id = input(  
    "Enter Session ID: "  
)
```

```
print("\nAI Chatbot")  
print("Type 'exit' to stop")  
print("-----")
```

```
# =====
```

```
# CHAT LOOP
```

```
# =====
```

```
while True:
```

```
    user_input = input(  
        "\nYou: "  
    )
```

```
    if user_input.lower() == "exit":
```

```
        print("Chat ended.")
```

```
        break
```

```
    response = chatbot.invoke(  
        {
```

```
            "question": user_input  
        },
```



```
config={  
    "configurable": {  
        "session_id": session_id  
    }  
}  
)
```

```
print(  
    "AI:",  
    response.content  
)
```

4. Architecture of In-Memory History

User



session_id



get_session_history()



store dictionary



InMemoryChatMessageHistory



RunnableWithMessageHistory



LCEL Chain



AI Response

5. What Is store = {}?

store = {}

This dictionary keeps histories for different sessions.

Conceptually:

```
store = {  
    "durga": history1,  
    "ravi": history2  
}
```

So:

```
session_id = "durga"
```



Durga History

```
session_id = "ravi"
```



Ravi History

6. What Does `get_session_history()` Do?

```
def get_session_history(session_id):
```

```
    if session_id not in store:
```

```
        store[session_id] = (  
            InMemoryChatMessageHistory()  
        )
```

```
    return store[session_id]
```

Flow:

```
session_id
```



Check store



History exists?

/ \

No

Yes



Create

Reuse

History

History

\ /



Return History

7. Important Limitation

This history exists in application memory.

So:

Program Running



History Available

but:

Program Stops



In-Memory Store Is Lost

Therefore:

InMemoryChatMessageHistory

≠

Permanent Storage

8. Approach 2 — Persistent File-Based Conversation History

Now we want:

Program Stops



History Still Exists



Program Starts Again



Previous History Can Be Loaded

For this, we create our own:

FileChatMessageHistory

9. Complete Persistent Chatbot Program

```
import os
```

```
import json
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import (
```

```
    ChatPromptTemplate,
```

```
    MessagesPlaceholder
```

```
)
```

```
from langchain_core.chat_history import (
```

```
    BaseChatMessageHistory
```

```
)
```

```
from langchain_core.messages import (
```

```
    BaseMessage,
```

```
    message_to_dict,
```

```
    messages_from_dict
```

```
)
```

```
from langchain_core.runnables.history import (
    RunnableWithMessageHistory
)

# =====
# 1. PERSISTENT CHAT HISTORY
# =====

class FileChatMessageHistory(
    BaseChatMessageHistory
):

    def __init__(
        self,
        session_id,
        storage_path="chat_history"
    ):

        self.session_id = session_id
```

```
os.makedirs(  
    storage_path,  
    exist_ok=True  
)
```

```
self.file_path = os.path.join(  
    storage_path,  
    f"{session_id}.json"  
)
```

```
# -----
```

```
# LOAD MESSAGES
```

```
# -----
```

```
@property
```

```
def messages(self):
```

```
    if not os.path.exists(  

```



```
        self.file_path
    ):
        return []

    with open(
        self.file_path,
        "r",
        encoding="utf-8"
    ) as file:

        data = json.load(file)

    return messages_from_dict(
        data
    )
```

```
# -----
```

```
# SAVE MESSAGES
```

```
# -----
```

```
def add_messages(  
    self,  
    messages: list[BaseMessage]  
) -> None:  
  
    all_messages = list(  
        self.messages  
    )  
  
    all_messages.extend(  
        messages  
    )  
  
    serialized = [  
        message_to_dict(message)  
        for message in all_messages  
    ]  
  
    with open(  

```

```
        self.file_path,  
        "w",  
        encoding="utf-8"  
    ) as file:
```

```
        json.dump(  
            serialized,  
            file,  
            ensure_ascii=False,  
            indent=2  
        )
```

```
# -----
```

```
# CLEAR HISTORY
```

```
# -----
```

```
def clear(self) -> None:
```

```
    with open(
```

```
        self.file_path,  
        "w",  
        encoding="utf-8"  
    ) as file:
```

```
        json.dump(  
            [],  
            file  
        )
```

```
# =====
```

```
# 2. CREATE MODEL
```

```
# =====
```

```
llm = ChatOpenAI(  
    model="gpt-5.6-luna",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
# =====
```

```
# 3. CREATE PROMPT
```

```
# =====
```

```
prompt = ChatPromptTemplate.from_messages(  
    [  
        (  
            "system",  
            """  
            You are a helpful AI assistant.  
            Answer in simple English.  
            """  
        ),  
        MessagesPlaceholder(  
            variable_name="history"  
        ),  
        (  

```

```
        "human",
        "{question}"
    )
]
)
```

```
# =====
```

```
# 4. CREATE LCEL CHAIN
```

```
# =====
```

```
chain = (
    prompt
    | llm
)
```

```
# =====
```

```
# 5. GET SESSION HISTORY
```

```
# =====
```

```
def get_session_history(
```

```
    session_id
```

```
):
```

```
    return FileChatMessageHistory(
```

```
        session_id
```

```
)
```

```
# =====
```

```
# 6. ADD MESSAGE HISTORY
```

```
# =====
```

```
chatbot = RunnableWithMessageHistory(
```

```
    chain,
```

```
    get_session_history,
```

```
input_messages_key="question",

history_messages_key="history"
)

# =====
# 7. GET SESSION ID
# =====

session_id = input(
    "Enter session id: "
)

print("\nAI Chatbot")
print("Type 'exit' to stop")
print("-----")
```



```
# =====
```

```
# 8. CHAT LOOP
```

```
# =====
```

```
while True:
```

```
    user_input = input(  
        "\nYou: "  
    )
```

```
    if user_input.lower() == "exit":
```

```
        print(  
            "Chat ended."  
        )
```

```
        break
```

```
response = chatbot.invoke(  
  
    {  
        "question": user_input  
    },  
  
    config={  
        "configurable": {  
            "session_id": session_id  
        }  
    }  
)
```

```
print(  
    "AI:",  
    response.content  
)
```

10. Main Difference Between the Two Programs

In-Memory version:

```
store = {}
```

and:

```
InMemoryChatMessageHistory()
```

Persistent version:

```
FileChatMessageHistory(  
    session_id  
)
```

which stores history in:

JSON File

11. File-Based Architecture

User



session_id



get_session_history()



FileChatMessageHistory



session_id.json



Load History



RunnableWithMessageHistory



LCEL Chain



AI Response



Save Updated History



session_id.json

12. One Session — One File

Suppose:

session_id = "durga"

Then:

chat_history/durga.json

Suppose:

```
session_id = "ravi"
```

Then:

```
chat_history/ravi.json
```

So:

Different Session IDs



Different JSON Files

13. Persistent History Folder

Conceptually:

```
chat_history/
```

```
|
```

```
├── durga.json
```

```
├── ravi.json
```

```
├── suresh.json
```

```
└── student1.json
```

14. What Is BaseChatMessageHistory?

We define:

```
class FileChatMessageHistory(
```

BaseChatMessageHistory

):

This means our custom class follows the chat-history structure expected by LangChain.

Our implementation provides:

messages

add_messages()

clear()

15. messages Property

@property

def messages(self):

This is responsible for loading old history.

We access it like:

history.messages

not:

history.messages()

16. Loading Messages

First:

```
if not os.path.exists(  
    self.file_path  
):  
    return []
```

Meaning:

File Does Not Exist



New Conversation



Return Empty History

17. Read JSON File

```
with open(  
    self.file_path,  
    "r",  
    encoding="utf-8"  
) as file:
```

```
data = json.load(file)
```

Here:

```
json.load(file)
```

means:

JSON File



Python Data

18. Restore LangChain Messages

```
return messages_from_dict(
```

```
    data
```

```
)
```

Flow:

JSON File



```
json.load()
```



Python Dictionaries



```
messages_from_dict()
```




HumanMessage / AIMessage

19. add_messages()

```
def add_messages(  
    self,  
    messages: list[BaseMessage]  
) -> None:
```

This method stores new messages.

First:

```
all_messages = list(  
    self.messages  
)
```

loads existing history.

Then:

```
all_messages.extend(  
    messages  
)
```

adds new messages.

20. Why extend()?

Suppose:

```
old = [m1, m2]
```

```
new = [m3, m4]
```

Using:

```
old.extend(new)
```

gives:

```
[m1, m2, m3, m4]
```

This is what we want.

21. Serialize Messages

```
serialized = [  
    message_to_dict(message)  
    for message in all_messages  
]
```

This converts:

LangChain Message Objects



Python Dictionaries

22. Save to JSON

```
json.dump(  
    serialized,  
    file,  
    ensure_ascii=False,  
    indent=2  
)
```

This means:

Python Data



JSON File

23. Save Flow

LangChain Messages



message_to_dict()



Python Dictionaries



json.dump()



JSON File

24. Load Flow

JSON File



`json.load()`



Python Dictionaries



`messages_from_dict()`



LangChain Messages

25. Clear History

`def clear(self) -> None:`

Inside:

`json.dump(`

`[],`

`file`

)

So:

Existing History



clear()



[]

The file remains, but conversation data becomes empty.

26. In-Memory get_session_history()

In the first program:

```
def get_session_history(session_id):
```

```
    if session_id not in store:
```

```
        store[session_id] = (
```

```
            InMemoryChatMessageHistory()
```

```
        )
```

```
    return store[session_id]
```

This uses:

Python Dictionary

as the session store.

27. Persistent get_session_history()

In the second program:

```
def get_session_history(
```

```
    session_id
```

```
):
```

```
    return FileChatMessageHistory(
```

```
        session_id
```

```
)
```

This uses:

session_id



JSON File

So no separate Python store = {} is required in this version.

28. Very Important Difference

In-Memory

session_id



store dictionary



InMemoryChatMessageHistory

Persistent

session_id



FileChatMessageHistory



session_id.json

29. RunnableWithMessageHistory Is Same in Both

In both programs we use:

```
chatbot = RunnableWithMessageHistory(  
    chain,  
    get_session_history,  
    input_messages_key="question",  
    history_messages_key="history"
```

)

The wrapper remains almost the same.

What changes is:

How Conversation History Is Stored

30. Important Observation

RunnableWithMessageHistory does not care whether history comes from:

Memory

File

Database

Other Storage

It asks:

`get_session_history(session_id)`

for the appropriate history object.

This separation is very important.

31. History Storage Is Replaceable

Concept:

RunnableWithMessageHistory



`get_session_history()`



History Implementation

Today:

`InMemoryChatMessageHistory`

or:

`FileChatMessageHistory`

Later it could be some other persistent storage.

32. Same Prompt in Both Programs

Both use:

```
MessagesPlaceholder(  
    variable_name="history"  
)
```

This is where previous conversation messages are inserted.

Prompt architecture:

System Message



Previous History



Current Question

33. Same LCEL Chain

Both programs use:

```
chain = (  
    prompt  
    | llm  
)
```

So:

Prompt



Model

remains unchanged.

34. Same Session Logic

Both use:

```
config={  
    "configurable": {  
        "session_id": session_id
```

```
}  
}
```

The session ID decides which history should be used.

35. Persistence Demonstration

This is the most important difference.

In-Memory Version

Run program:

Session: durga

You: My name is Durga.

Stop program.

Run again.

The old Python dictionary does not automatically remain.

File-Based Version

Run program:

Session: durga

You: My name is Durga.

This conversation is saved into:

chat_history/durga.json

Stop program.

Start again.

Use:

Session: durga

Ask:

What is my name?

Previous file data can be loaded again.

36. In-Memory vs Persistent Comparison

Concept In-Memory		File-Based Persistent
History class	InMemoryChatMessageHistory	FileChatMessageHistory
Storage	Application memory	JSON file
Python dictionary needed	Yes in this design	No
Survives restart	No, not automatically	Yes

Separate sessions	Yes	Yes
Session identifier	session_id	session_id
Wrapper	RunnableWithMessageHistory	RunnableWithMessageHistory
Prompt	Same	Same
LCEL chain	Same	Same

37. Main Code Difference

In-Memory

```
store = {}
```

```
def get_session_history(session_id):
```

```
    if session_id not in store:
```

```
        store[session_id] = (
```

```
        InMemoryChatMessageHistory()  
    )
```

```
    return store[session_id]
```

File-Based

```
def get_session_history(  
    session_id  
):
```

```
    return FileChatMessageHistory(  
        session_id  
    )
```

38. Why Is the Persistent Version More Complex?

Because it must handle:

Folder Creation

File Path

File Existence

Reading JSON

Writing JSON

Serialization

Deserialization

Clearing Data

The in-memory version does not need all these operations.

39. Important JSON Functions

`json.load()`



JSON File → Python

`json.dump()`



Python → JSON File

For LangChain messages:

message_to_dict()



Message → Dictionary

messages_from_dict()



Dictionary → Message

40. Complete Persistent Cycle

Current User Message



RunnableWithMessageHistory



Get Session History



Read JSON File



Deserialize Messages



Previous History

+

Current Question



Model



AI Response



Serialize Updated Messages



Save JSON File

41. Five Important Similarities

Both programs use:

1. ChatOpenAI
2. ChatPromptTemplate
3. MessagesPlaceholder
4. LCEL chain
5. RunnableWithMessageHistory
6. session_id

The major difference is only:

History Storage Mechanism

42. Five Important Differences

1. Storage

In-Memory



RAM / Application Memory

Persistent



JSON File

2. Restart

In-Memory



History Lost

Persistent



History Reloaded

3. History Class

InMemoryChatMessageHistory

versus:

FileChatMessageHistory

4. Serialization

In-memory program does not manually need:

`message_to_dict()`

`messages_from_dict()`

File version does.

5. File Handling

Persistent version needs:

`open()`

`json.load()`

`json.dump()`

`os.path.exists()`

43. Common Mistake — Thinking Session ID Gives Persistence

Wrong.

`session_id`

`≠`

Persistence

`session_id` only identifies the conversation.

Persistence depends on the storage mechanism.

44. Common Mistake — Thinking RunnableWithMessageHistory Stores Permanently

Wrong.

RunnableWithMessageHistory

≠

Permanent Database

It manages history around the Runnable.

Actual persistence depends on the chat-history implementation.

45. Common Mistake — Thinking InMemoryChatMessageHistory Saves Files

Wrong.

InMemoryChatMessageHistory

↓

Application Memory

It does not automatically create JSON files.

46. Common Mistake — Thinking FileChatMessageHistory Is Built-In Here

In this example:

FileChatMessageHistory

is our own custom class.

We created it by extending:

BaseChatMessageHistory

47. Most Important Design Principle

Separate:

Conversation Logic

from:

History Storage

The chatbot logic can stay almost the same.

Only the history provider changes.

48. Architecture Comparison

In-Memory

USER



session_id



get_session_history()



store dictionary



InMemoryChatMessageHistory



RunnableWithMessageHistory



LCEL Chain



Model

Persistent

USER



session_id



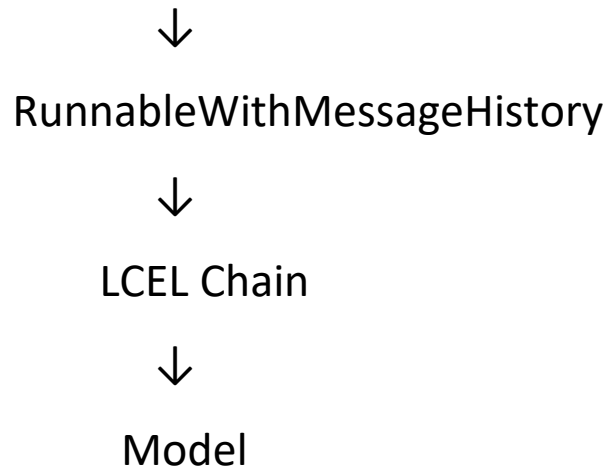
get_session_history()



FileChatMessageHistory



session_id.json



49. Important Key Points

1. InMemoryChatMessageHistory stores history only in application memory.
2. A Python dictionary can maintain separate in-memory histories for separate sessions.
3. FileChatMessageHistory stores each session in a JSON file.
4. BaseChatMessageHistory is used as the base for our custom file history.
5. `message_to_dict()` serializes LangChain messages.
6. `messages_from_dict()` restores LangChain messages.
7. `json.dump()` writes data to the JSON file.
8. `json.load()` reads data from the JSON file.
9. `session_id` identifies which conversation history should be used.

10. Same session ID can continue the same conversation.
 11. Different session IDs can maintain separate conversations.
 12. `RunnableWithMessageHistory` works with both in-memory and persistent history implementations.
 13. Persistence depends on the storage layer, not on `session_id` itself.
 14. File-based persistence allows conversation history to survive a program restart.
 15. The prompt and LCEL chain can remain the same even when we change the storage mechanism.
-

50. MCQ 1

What is the main limitation of:

`InMemoryChatMessageHistory()`

- A. It cannot store messages
- B. It is not automatically persistent after program restart
- C. It cannot use session IDs
- D. It cannot work with `Runnables`

Answer: B

51. MCQ 2

What is the main purpose of:

FileChatMessageHistory

in this program?

- A. Generate embeddings
- B. Store conversation history in JSON files
- C. Replace the LLM
- D. Parse structured output

Answer: B

52. MCQ 3

What decides whether chat history survives program restart?

- A. session_id alone
- B. MessagesPlaceholder
- C. The history storage implementation
- D. ChatPromptTemplate

Answer: C

53. MCQ 4

Which function converts LangChain messages into dictionaries?

- A. messages_from_dict()
- B. message_to_dict()
- C. json.load()
- D. os.path.join()

Answer: B

54. MCQ 5

Which statement is correct?

- A. RunnableWithMessageHistory automatically means persistent database storage.
- B. In-memory and file-based history cannot use the same Runnable wrapper.
- C. The same RunnableWithMessageHistory architecture can work with different history implementations.
- D. Session IDs are not required for multiple conversations.

Answer: C

55. Final Master Formula

RunnableWithMessageHistory

+

get_session_history()

↓

History-Aware Chatbot

What changes is:

get_session_history()

↓

In-Memory History

or:

`get_session_history()`



Persistent File History

56. Final Memory Line

IN-MEMORY

=====

Session



Python Store



Temporary History

PERSISTENT

=====

Session



JSON File



57. One-Line Conclusion

The chatbot architecture stays almost the same in both programs; the main difference is that `InMemoryChatMessageHistory` keeps history only in application memory, while our custom `FileChatMessageHistory` stores each session in a JSON file so the conversation can be reco